

02393 Programming in C++

Module 11: Linked Lists

© Giovanni Meroni

Slides based on previous versions by Alceste Scalas, Andrea Vandin, Alberto Lluch Lafuente, Sebastian Mödersheim

Lecture plan

Module	Date	Title	Textbook chapters
0 and 1	03/09	Welcome & C++ Overview	1
2	10/09	Basics of C++ and its Data Types	1.5, 2.1, 2.3 - 2.5, 11.1, 11.3
3	17/09	Memory management	12.1, 11.2, 11.3
4	24/09	LAB DAY	N/A
5	01/10	Libraries and Interfaces	2.2, 2.6 - 2.8, 3.1 - 3.3, 4.1 - 4.3
6	08/10	Classes and Objects	5.1, 6.1, 11.2, 12.1, 12.4, 12.7
Autumn break			
7	22/10	Templates	5.1, 14.2
8	29/10	Inheritance	19.1 - 19.3
9	05/11	LAB DAY	N/A
10	12/11	Recursive Programming	7
11	19/11	Linked Lists	12.2, 12.3
12	26/11	Trees	16.1 - 16.4
13	03/12	LAB DAY	N/A

Outline

- Recap
- Single-linked lists
- Doubly-linked lists
- Lab

A recap from the previous lectures

- The structure of a C++ program
 - `#include` and `#define` directives, the main function, user-defined functions and methods (including constructors, destructors, operators), templates
- Simple input/output
 - `cin`, `cout`
- Variables, values and types
 - `string`, `int`, `double`, `float`, `bool`, arrays (statically and dynamically allocated), pointers, `enum`, `struct`, `vector`, `ifstream`, `ofstream`, `class`, `this`
- Expressions
 - some numeric and boolean operators and math functions, conditional expressions
- Statements
 - `if`, `while`, `for`, `switch`
- Recursive programming
 - Recursive functions

Recursion: rules of thumb

- Identify the base cases (i.e., the “smallest” cases whose solution does not need recursion)
- Solve the base cases
- Address the recursive cases (i.e., the non-base cases that need recursive calls)
 - Ensure that the arguments to the recursive calls are “smaller” than the original arguments
 - Ensure that recursive calls eventually reach one of the base cases

Recursive data types

- Many mathematical entities can be recursively defined:
 - A natural number is either 0, or the successor of a natural number
 - A set can be either empty, or a singleton, or the union of two sets
- Similarly, many data types can be recursively defined:
 - A list can be either empty, or one item, or the concatenation of two lists
 - A tree can be either empty, or one leaf node, or an internal node with two sub-trees

Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 class Node {  
7 public:  
8     int content;  
9     Node *next;  
10 };
```

- A Node instance has some content and points to the next Node in the list
 - Nodes can be implemented as structures or classes
- A linked list is usually dynamically allocated, and nodes may be added or removed
- Therefore, a linked list is accessed through a pointer to a Node (i.e., Node*)
 - The pointer nullptr represents an empty list
 - Otherwise, it is a pointer to the first node of the list
- Important: do not forget to initialize your pointers!

Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 Node* findNodeIterative(Node *head, int value) {  
7     while (head != nullptr){  
8         if (head->content == value) return head;  
9         head = head->next;  
10    }  
11    return nullptr;  
12 }  
13  
14 Node* findNodeRecursive(Node *currNode, int value) {  
15     if (currNode == nullptr){  
16         return nullptr;  
17     } else if (currNode->content == value){  
18         return currNode;  
19     } else {  
20         return findNodeRecursive(currNode->next, value);  
21     }  
22 }
```

- To find nodes in a list, one has to:
 - Check if the end of the list has been reached

Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 Node* findNodeIterative(Node *head, int value) {  
7     while (head != nullptr){  
8         if (head->content == value) return head;  
9         head = head->next;  
10    }  
11    return nullptr;  
12 }  
13  
14 Node* findNodeRecursive(Node *currNode, int value) {  
15     if (currNode == nullptr){  
16         return nullptr;  
17     } else if (currNode->content == value){  
18         return currNode;  
19     } else {  
20         return findNodeRecursive(currNode->next, value);  
21     }  
22 }
```

- To find nodes in a list, one has to:
 - Check if the end of the list has been reached
 - Check if the current node contains the specified value

Single-linked lists

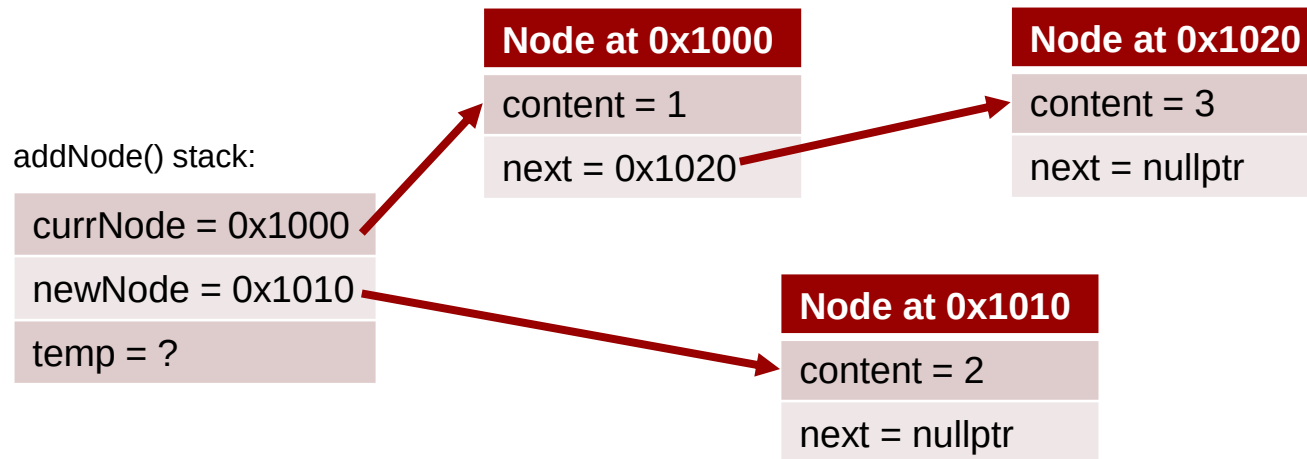
```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 Node* findNodeIterative(Node *head, int value) {  
7     while (head != nullptr){  
8         if (head->content == value) return head;  
9         head = head->next;  
10    }  
11    return nullptr;  
12 }  
13  
14 Node* findNodeRecursive(Node *currNode, int value) {  
15     if (currNode == nullptr){  
16         return nullptr;  
17     } else if (currNode->content == value){  
18         return currNode;  
19     } else {  
20         return findNodeRecursive(currNode->next, value);  
21     }  
22 }
```

- To find nodes in a list, one has to:
 - Check if the end of the list has been reached
 - Check if the current node contains the specified value
 - Move to the next node
- Once in the next node, previous nodes can no longer be reached

Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void addNode(Node *currNode, Node *newNode){  
7     Node *temp = currNode->next;  
8     currNode->next = newNode;  
9     newNode->next = temp;  
10 }
```

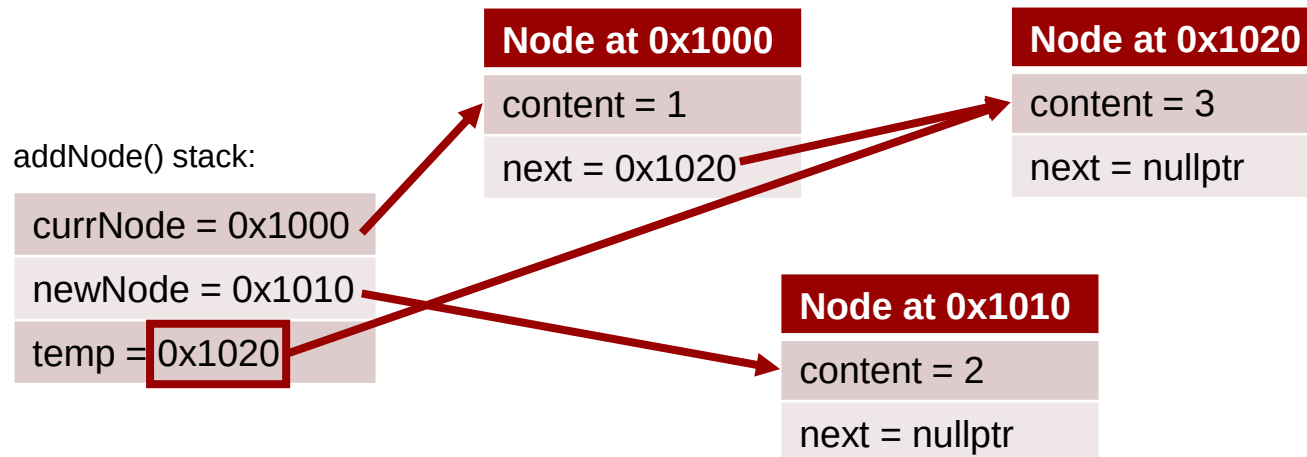
- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted



Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void addNode(Node *currNode, Node *newNode){  
7     Node *temp = currNode->next;  
8     currNode->next = newNode;  
9     newNode->next = temp;  
10 }
```

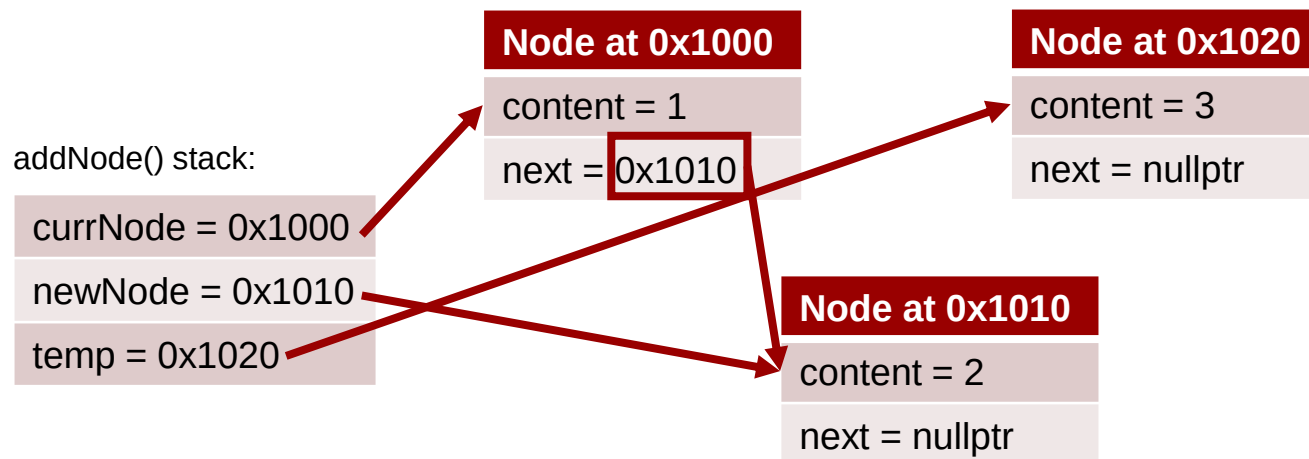
- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node



Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void addNode(Node *currNode, Node *newNode){  
7     Node *temp = currNode->next;  
8     currNode->next = newNode;  
9     newNode->next = temp;  
10 }
```

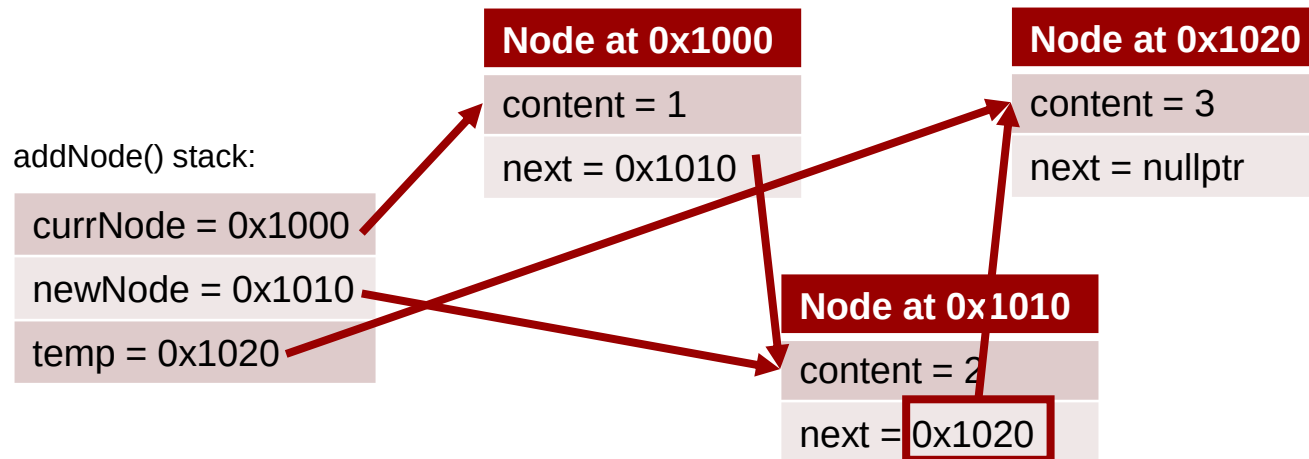
- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node
 - Assign the address of the new node to the pointer of the next node of the current node



Single-linked lists

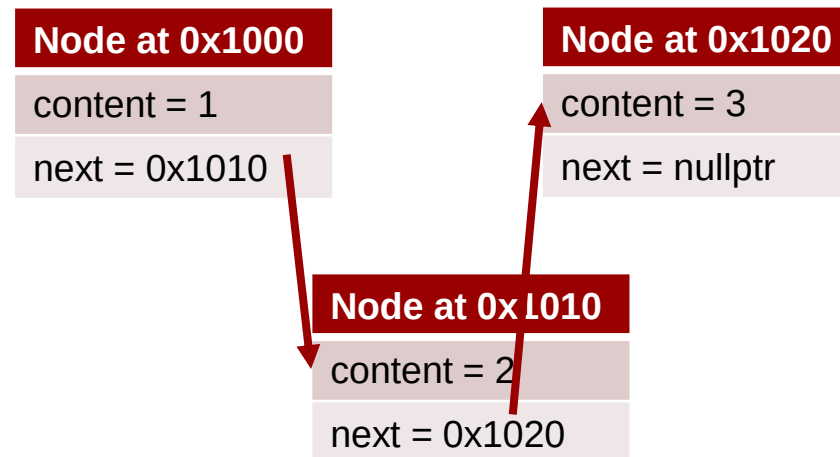
```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void addNode(Node *currNode, Node *newNode){  
7     Node *temp = currNode->next;  
8     currNode->next = newNode;  
9     newNode->next = temp;  
10 }
```

- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node
 - Assign the address of the new node to the pointer of the next node of the current node
- Assign the address of the old next node to the pointer to the next node in the new node



Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void addNode(Node *currNode, Node *newNode){  
7     Node *temp = currNode->next;  
8     currNode->next = newNode;  
9     newNode->next = temp;  
10 }
```



- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node
 - Assign the address of the new node to the pointer of the next node of the current node
- Assign the address of the old next node to the pointer to the next node in the new node

Single-linked lists

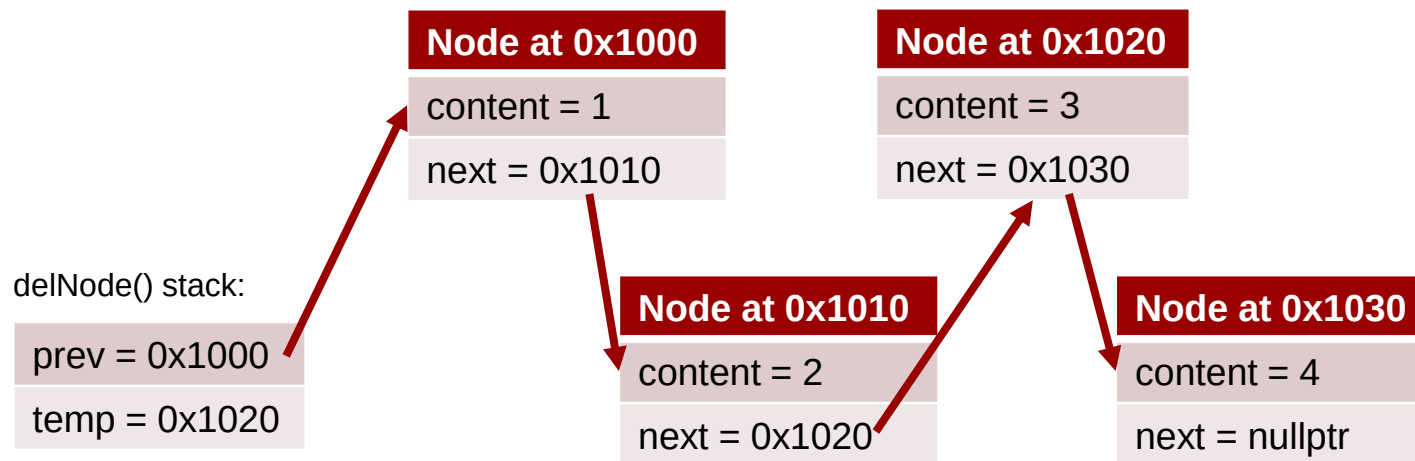
```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void addNode(Node *currNode, Node *newNode){  
7     Node *temp = currNode->next;  
8     currNode->next = newNode;  
9     newNode->next = temp;  
10 }
```

- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node
 - Assign the address of the new node to the pointer of the next node of the current node
 - Assign the address of the old next node to the pointer to the next node in the new node
- Special cases (not in this code):
 - The list is empty
 - We need to add a new node in front of the list

Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void delNode(Node *prev) {  
7     Node temp = prev->next->next;  
8     delete prev->next;  
9     prev->next = temp;  
10 }
```

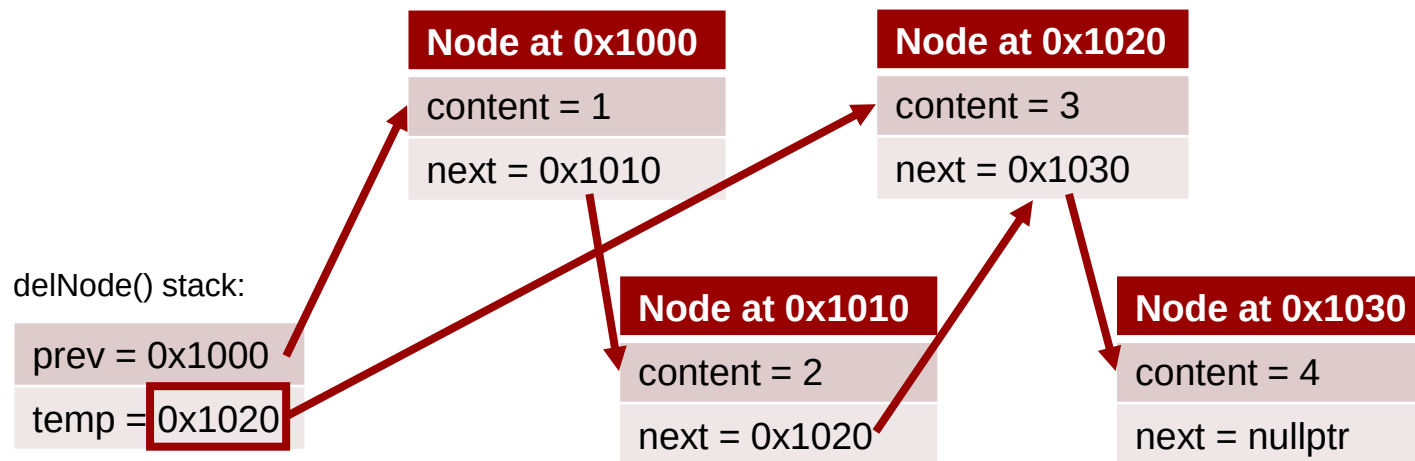
- When deleting a node from a list, one needs to:
 - Locate the predecessor of that node



Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void delNode(Node *prev) {  
7     Node temp = prev->next->next;  
8     delete prev->next;  
9     prev->next = temp;  
10 }
```

- When deleting a node from a list, one needs to:
 - Locate the predecessor of that node
 - Save the address of the successor of the node to be deleted



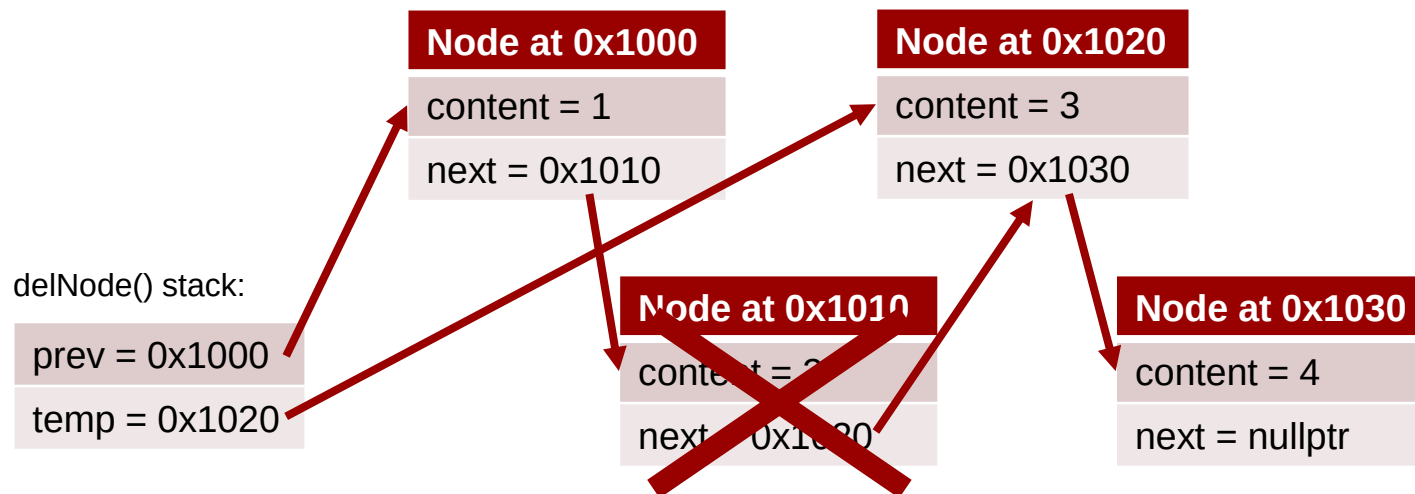
Single-linked lists

```

1 struct Node {
2     int content;
3     Node *next;
4 }
5
6 void delNode(Node *prev) {
7     Node temp = prev->next->next;
8     delete prev->next;
9     prev->next = temp;
10 }

```

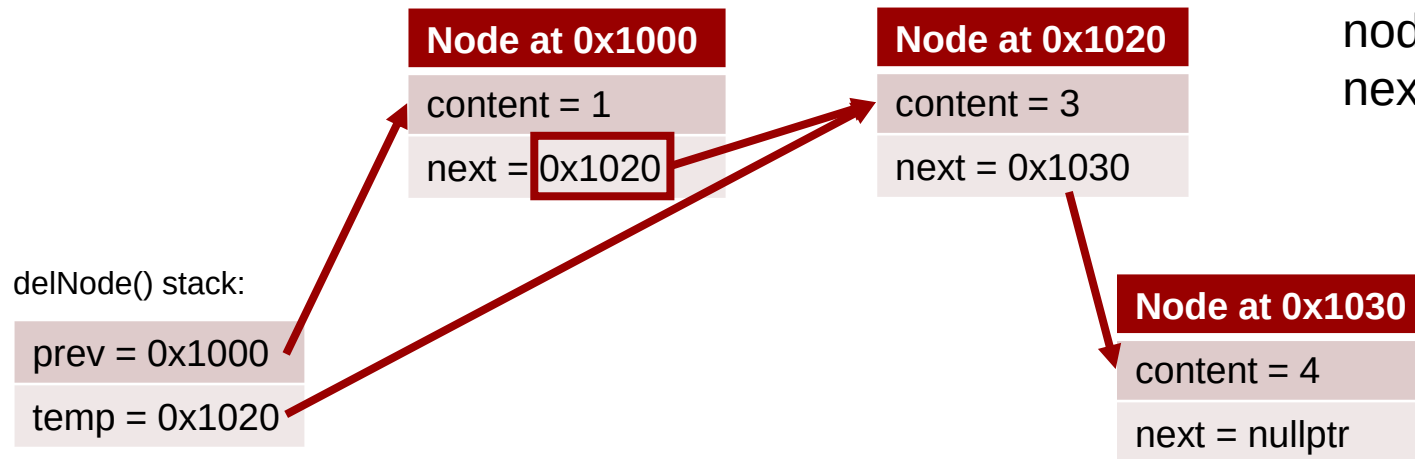
- When deleting a node from a list, one needs to:
 - Locate the predecessor of that node
 - Save the address of the successor of the node to be deleted
 - Deallocate the memory used by the node to be deleted



Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void delNode(Node *prev) {  
7     Node temp = prev->next->next;  
8     delete prev->next;  
9     prev->next = temp;  
10 }
```

- When deleting a node from a list, one needs to:
 - Locate the predecessor of that node
 - Save the address of the successor of the node to be deleted
 - Deallocate the memory used by the node to be deleted
 - Assign the address of the successor of node to be deleted to the pointer to the next node of the predecessor



Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void delNode(Node *prev) {  
7     Node temp = prev->next->next;  
8     delete prev->next;  
9     prev->next = temp;  
10 }
```

Node at 0x1000

content = 1

next = 0x1020

Node at 0x1020

content = 3

next = 0x1030

Node at 0x1030

content = 4

next = nullptr

- When deleting a node from a list, one needs to:
 - Locate the predecessor of that node
 - Save the address of the successor of the node to be deleted
 - Deallocate the memory used by the node to be deleted
 - Assign the address of the successor of node to be deleted to the pointer to the next node of the predecessor

Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void delNode(Node *prev) {  
7     Node temp = prev->next->next;  
8     delete prev->next;  
9     prev->next = temp;  
10 }
```

- When deleting a node from a list, one needs to:
 - Locate the predecessor of that node
 - Save the address of the successor of the node to be deleted
 - Deallocate the memory used by the node to be deleted
 - Assign the address of the successor of node to be deleted to the pointer to the next node of the predecessor
- Special cases (not in this code):
 - The node to be deleted is the first
 - The list has only one node
 - The list is empty

Single-linked lists

- In previous lectures we saw an example of how to implement a vector-like class based on arrays
- Today we are going to see how to implement:
 - A vector-like class based on linked lists
- In today's lecture code on DTU Learn you will also find:
 - A set-like class based on linked lists
- Note: the C++ STL provides linked lists. See:
<http://en.cppreference.com/w/cpp/container/list>

Doubly-linked lists

- Some annoying tasks with single-linked lists:
 - Delete a pointed element (how do we update the previous element?)
 - Concatenate two lists (time complexity $O(N)$, as we need to reach the end of the first list)
- One possible solution: doubly-linked lists

Doubly-Linked Lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4     Node *prev  
5 }  
6  
7 class Node {  
8 public:  
9     int content;  
10    Node *next;  
11    Node *prev;  
12 };
```

- Node instance has some content and points to the next and previous Nodes in the list
 - The first element in the list will have prev set to nullptr
 - The last element in the list will have next set to nullptr

Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void addNode(Node *currNode, Node *newNode){
7     Node *temp = currNode->next;
8     currNode->next = newNode;
9     newNode->next = temp;
10    newNode->prev = currNode;
11    if(temp!=nullptr) temp->prev = newNode;
12 }

```

- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted

addNode() stack:

currNode = 0x1000

newNode = 0x1010

temp = ?

Node at 0x1000

content = 1

next = 0x1020

prev = nullptr

Node at 0x1020

content = 3

next = nullptr

prev = 0x1000

Node at 0x1010

content = 2

next = nullptr

prev = nullptr

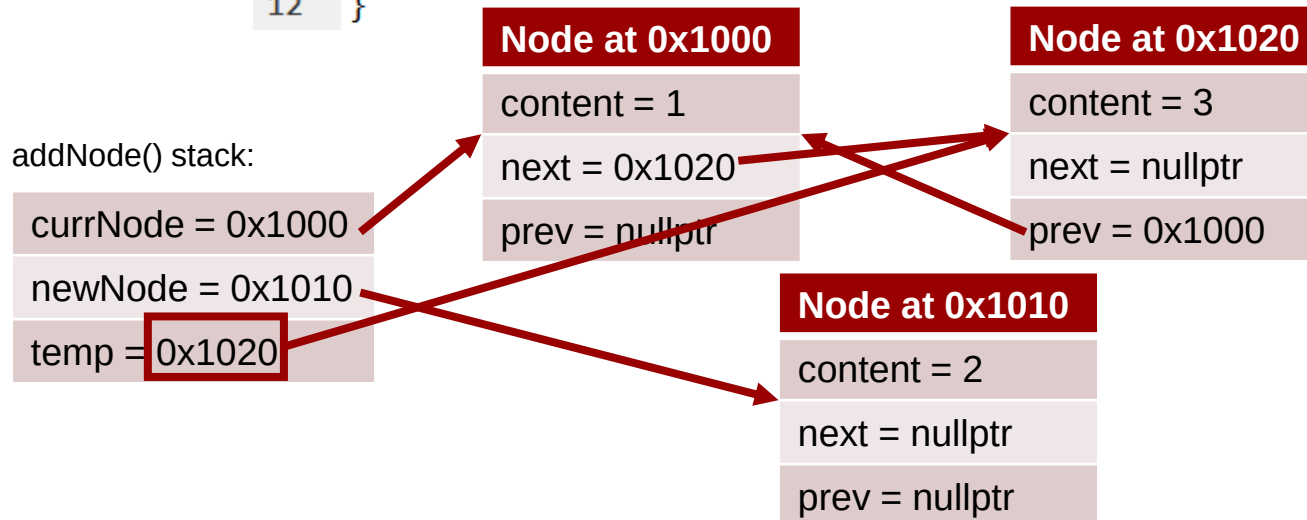
Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void addNode(Node *currNode, Node *newNode){
7     Node *temp = currNode->next;
8     currNode->next = newNode;
9     newNode->next = temp;
10    newNode->prev = currNode;
11    if(temp!=nullptr) temp->prev = newNode;
12 }

```

- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node



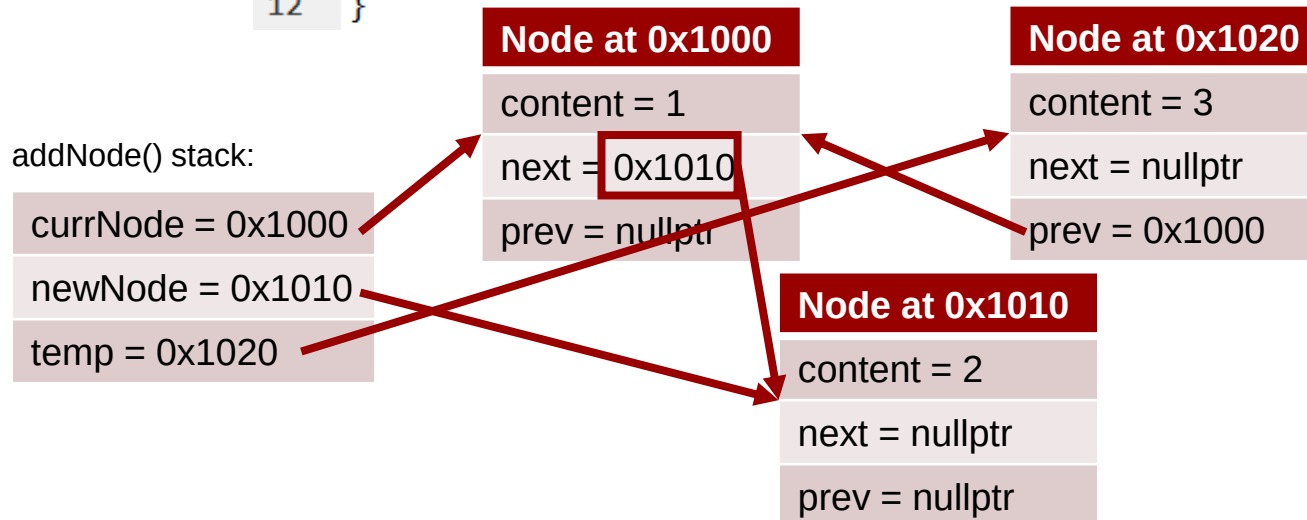
Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void addNode(Node *currNode, Node *newNode){
7     Node *temp = currNode->next;
8     currNode->next = newNode;
9     newNode->next = temp;
10    newNode->prev = currNode;
11    if(temp!=nullptr) temp->prev = newNode;
12 }

```

- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node
 - Assign the address of the new node to the pointer of the next node of the current node



Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void addNode(Node *currNode, Node *newNode){
7     Node *temp = currNode->next;
8     currNode->next = newNode;
9     newNode->next = temp;
10    newNode->prev = currNode;
11    if(temp!=nullptr) temp->prev = newNode;
12 }

```

- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node
 - Assign the address of the new node to the pointer of the next node of the current node
 - Assign the address of the new node to the pointer of the next node in the current node

addNode() stack:

currNode = 0x1000

newNode = 0x1010

temp = 0x1020

Node at 0x1000

content = 1

next = 0x1010

prev = nullptr

Node at 0x1020

content = 3

next = nullptr

prev = 0x1000

Node at 0x1010

content = 2

next = 0x1020

prev = nullptr

Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void addNode(Node *currNode, Node *newNode){
7     Node *temp = currNode->next;
8     currNode->next = newNode;
9     newNode->next = temp;
10    newNode->prev = currNode;
11    if(temp!=nullptr) temp->prev = newNode;
12 }

```

addNode() stack:

currNode = 0x1000

newNode = 0x1010

temp = 0x1020

Node at 0x1000

content = 1

next = 0x1010

prev = nullptr

Node at 0x1020

content = 3

next = nullptr

prev = 0x1000

Node at 0x1010

content = 2

next = 0x1020

prev = 0x1000

- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node
 - Assign the address of the new node to the pointer of the next node of the current node
 - Assign the address of the new node to the pointer of the next node in the current node
 - **Assign the address of the current node to the pointer of the previous node in the new node**

Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void addNode(Node *currNode, Node *newNode){
7     Node *temp = currNode->next;
8     currNode->next = newNode;
9     newNode->next = temp;
10    newNode->prev = currNode;
11    if(temp!=nullptr) temp->prev = newNode;
12 }

```

addNode() stack:

currNode = 0x1000

newNode = 0x1010

temp = 0x1020

Node at 0x1000

content = 1

next = 0x1010

prev = nullptr

Node at 0x1020

content = 3

next = nullptr

prev = 0x1010

Node at 0x1010

content = 2

next = 0x1020

prev = 0x1000

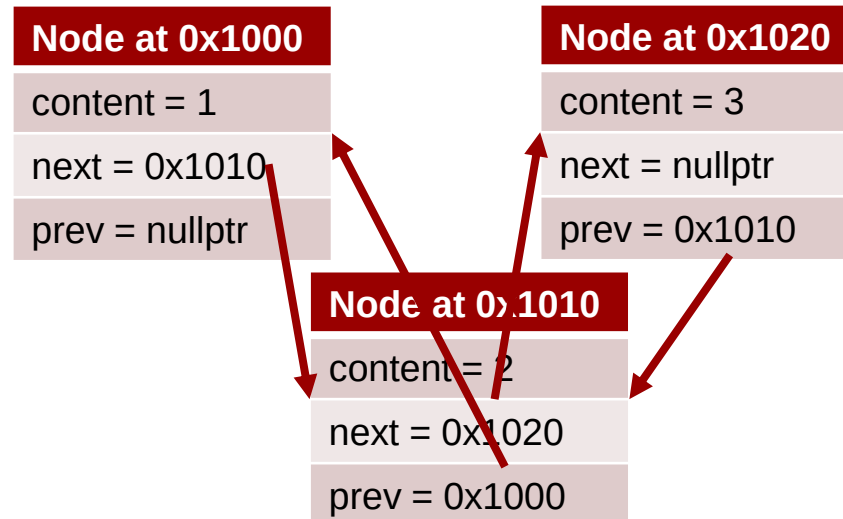
- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node
 - Assign the address of the new node to the pointer of the next node of the current node
 - Assign the address of the new node to the pointer of the next node in the current node
 - **Assign the address of the current node to the pointer of the previous node in the new node**
 - **Assign the address of the new node to the pointer of the previous node in the old next node (if it exists)**

Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void addNode(Node *currNode, Node *newNode){
7     Node *temp = currNode->next;
8     currNode->next = newNode;
9     newNode->next = temp;
10    newNode->prev = currNode;
11    if(temp!=nullptr) temp->prev = newNode;
12 }

```



- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node
 - Assign the address of the new node to the pointer of the next node of the current node
 - Assign the address of the new node to the pointer of the next node in the current node
 - **Assign the address of the current node to the pointer of the previous node in the new node**
 - **Assign the address of the new node to the pointer of the previous node in the old next node (if it exists)**

Doubly-Linked Lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4     Node *prev  
5 }  
6 void addNode(Node *currNode, Node *newNode){  
7     Node *temp = currNode->next;  
8     currNode->next = newNode;  
9     newNode->next = temp;  
10    newNode->prev = currNode;  
11    if(temp!=nullptr) temp->prev = newNode;  
12 }
```

- Special cases (not in this code):
 - The list is empty
 - We need to add a new node in front of the list

- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node
 - Assign the address of the new node to the pointer of the next node of the current node
 - Assign the address of the new node to the pointer of the next node in the current node
 - **Assign the address of the current node to the pointer of the previous node in the new node**
 - **Assign the address of the new node to the pointer of the previous node in the old next node (if it exists)**

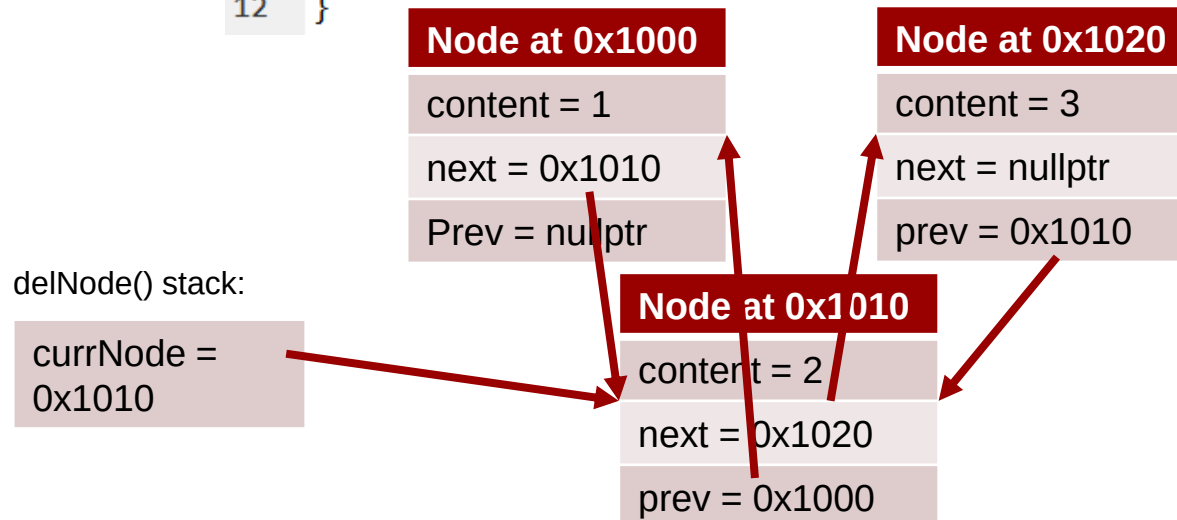
Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void delNode(Node *currNode) {
7     if(currNode->prev != nullptr)
8         currNode->prev->next = currNode->next;
9     if(currNode->next != nullptr)
10        currNode->next->prev = currNode->prev;
11    delete currNode;
12 }

```

- When deleting a node from a list, one needs to:
 - Locate the node to be deleted



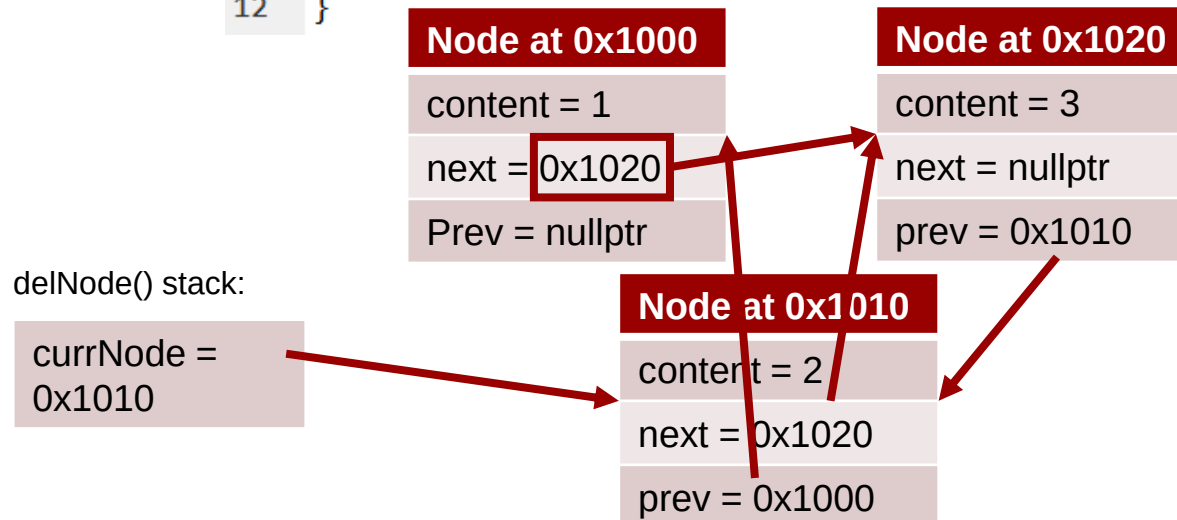
Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void delNode(Node *currNode) {
7     if(currNode->prev != nullptr)
8         currNode->prev->next = currNode->next;
9     if(currNode->next != nullptr)
10        currNode->next->prev = currNode->prev;
11    delete currNode;
12 }

```

- When deleting a node from a list, one needs to:
 - Locate the node to be deleted
 - Assign the address of the successor of node to be deleted to the pointer to the next node of the predecessor (if any)



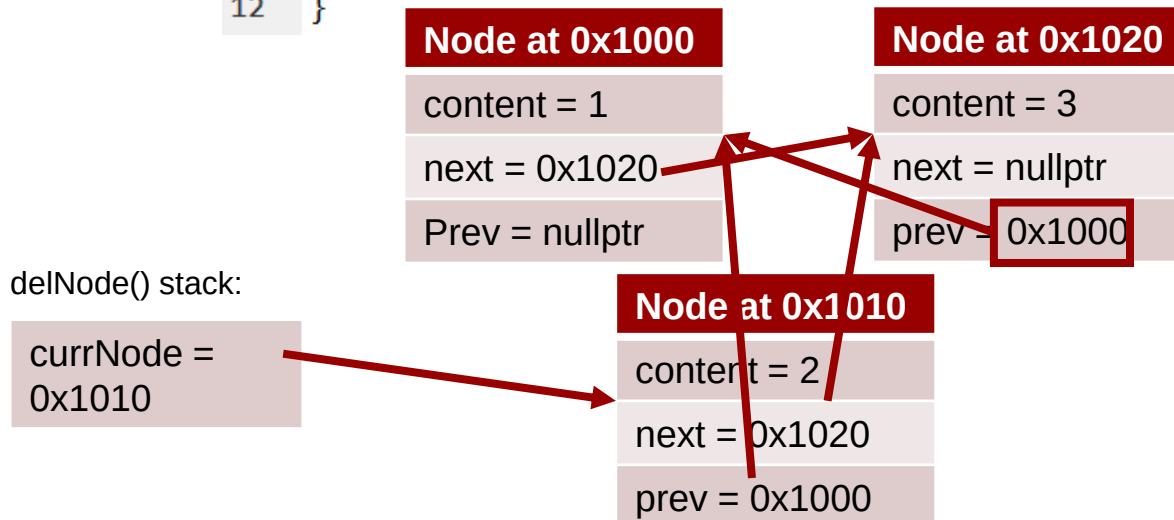
Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void delNode(Node *currNode) {
7     if(currNode->prev != nullptr)
8         currNode->prev->next = currNode->next;
9     if(currNode->next != nullptr)
10        currNode->next->prev = currNode->prev;
11    delete currNode;
12 }

```

- When deleting a node from a list, one needs to:
 - Locate the node to be deleted
 - Assign the address of the successor of node to be deleted to the pointer to the next node of the predecessor (if any)
 - Assign the address of the predecessor of the node to be deleted to the pointer to the previous node of the successor (if any)

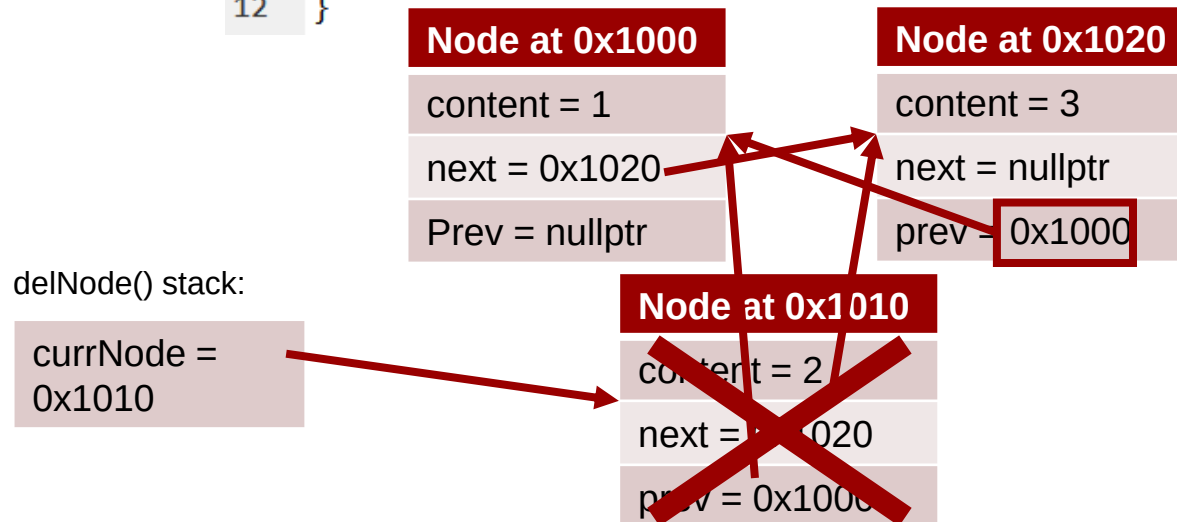


Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void delNode(Node *currNode) {
7     if(currNode->prev != nullptr)
8         currNode->prev->next = currNode->next;
9     if(currNode->next != nullptr)
10        currNode->next->prev = currNode->prev;
11     delete currNode;
12 }

```



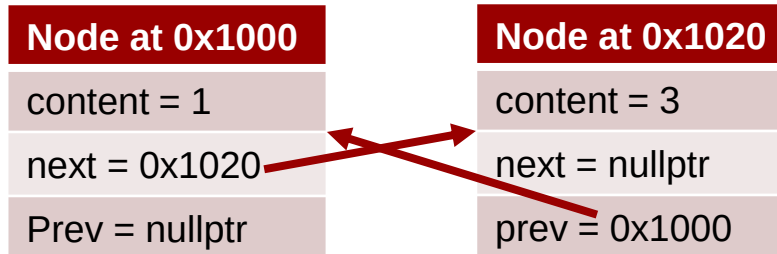
- When deleting a node from a list, one needs to:
 - Locate the node to be deleted
 - Assign the address of the successor of node to be deleted to the pointer to the next node of the predecessor (if any)
 - Assign the address of the predecessor of the node to be deleted to the pointer to the previous node of the successor (if any)
 - Deallocate the memory used by the node to be deleted

Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void delNode(Node *currNode) {
7     if(currNode->prev != nullptr)
8         currNode->prev->next = currNode->next;
9     if(currNode->next != nullptr)
10        currNode->next->prev = currNode->prev;
11     delete currNode;
12 }

```



- When deleting a node from a list, one needs to:
 - Locate the node to be deleted
 - Assign the address of the successor of node to be deleted to the pointer to the next node of the predecessor (if any)
 - Assign the address of the predecessor of the node to be deleted to the pointer to the previous node of the successor (if any)
 - Deallocate the memory used by the node to be deleted

Doubly-Linked Lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4     Node *prev  
5 }  
6 void delNode(Node *currNode) {  
7     if(currNode->prev != nullptr)  
8         currNode->prev->next = currNode->next;  
9     if(currNode->next != nullptr)  
10        currNode->next->prev = currNode->prev;  
11     delete currNode;  
12 }
```

- Special cases (not in this code):
 - The node to be deleted is the first
 - The node to be deleted is the last
 - The list has only one node
 - The list is empty

- When deleting a node from a list, one needs to:
 - Locate the node to be deleted
 - Assign the address of the successor of node to be deleted to the pointer to the next node of the predecessor (if any)
 - Assign the address of the predecessor of the node to be deleted to the pointer to the previous node of the successor (if any)
 - Deallocate the memory used by the node to be deleted

Lab time

- Today
 - Make sure C++ works on your computer, request help if it doesn't
 - Begin working on Assignment 10
 - Ask questions if something is unclear (including previous assignments)

DTU

